

PDL: A Declarative Prompt Programming Language

MANDANA VAZIRI*, LOUIS MANDEL*, CLAUDIO SPIESS†, and MARTIN HIRZEL*

Large language models (LLMs) have taken the world by storm by making many previously difficult uses of AI feasible. LLMs are controlled via highly expressive textual prompts and return textual answers. Unfortunately, this unstructured text as input and output makes LLM-based applications brittle. This motivates the rise of prompting frameworks, which mediate between LLMs and the external world. However, existing prompting frameworks either have a high learning curve or take away control over the exact prompts from the developer. To overcome this dilemma, this paper introduces the Prompt Declaration Language (PDL). PDL is a simple declarative *data-oriented* language that puts prompts at the forefront, based on YAML. PDL works well with many LLM platforms and LLMs. It supports writing interactive applications that call LLMs and tools, and makes it easy to implement common use-cases such as chatbots, RAG, or agents. We hope PDL will make prompt programming simpler, less brittle, and more enjoyable.

1 Introduction

Large language models (LLMs) have made great advances, demonstrating the ability to perform a wide range of useful tasks. As LLMs are controlled via natural-language *prompts*, prompt engineering has emerged as an ad-hoc approach to improve accuracy [30]. Even more capabilities can be unlocked with prompting patterns such as in-context learning [6], chaining multiple LLM calls [7], retrieval-augmented generation (RAG) [17], tool use [27], program-aided language models (PAL) [10], and agents [33]. Unfortunately, while powerful, LLMs remain brittle: they sometimes hallucinate, or even fail to comply with expected syntax and types.

Prompting frameworks [19] make it easier for developers to use LLMs and associated prompting patterns while ameliorating their brittleness. Some, such as LangChain [7] and AutoGen [31], do so via bespoke features for popular patterns such as RAG or agents. Unfortunately, this bespokeness takes control over basic prompts away from users and forces them to learn many complex framework features. In contrast, low-level prompting frameworks, such as Guidance [22] and LMQL [5], provide more control with syntax and types. Unfortunately, they require users to program in imperative languages such as Python or TypeScript. At the other end of the spectrum, frameworks such as DSPy [14] and Vieira [18] avoid hand-written prompts altogether by automatically generating them. Unfortunately, this takes away even more control from the developer. The problem thus becomes how to make LLM programming less brittle while keeping it simple and keeping the developer in the driver’s seat.

To tackle this problem, we turned to tried-and-true programming language design ideas. The principle of *orthogonality* advocates for a small set of simple features that compose in powerful ways [29]. Being orthogonal, or at right angles, here means being irredundant and avoiding exceptional cases as far as possible. For prompting frameworks, orthogonality is a way to avoid bespoke features. Next, developers need to struggle less with brittleness if the language checks *types* and *roles* [12] to enforce structure by construction. One remaining tension is harder to tackle: on the one hand, we want to give developers control over the exact prompts, but on the other hand, we want a simple *declarative* language. To this end, we settled on a *data-oriented language*, which puts prompts at the forefront by intentionally blurring the line between programs (e.g. for chaining and tools) and data (for prompts). This is inspired by the old idea of code as data [20], as well as by seminal work on programming without tiers [8].

*IBM Research

†UC Davis

PDL: 一种声明式提示编程语言

MANDANA VAZIRI*, LOUIS MANDEL*, CLAUDIO SPIESS†, and MARTIN HIRZEL*

大型语言模型 (LLMs) 通过使用许多先前难以实现的 AI 应用变得可行, 已在全球范围内掀起热潮。LLMs 通过高度表达性的文本提示进行控制, 并返回文本答案。不幸的是, 这种非结构化的文本作为输入和输出, 使得基于 LLM 的应用程序变得脆弱。这促使了提示框架的兴起, 这些框架在 LLMs 与外部世界之间起到中介作用。然而, 现有的提示框架要么学习曲线陡峭, 要么剥夺了开发者对具体提示的控制权。为了克服这一困境, 本文介绍了提示声明语言 (PDL)。PDL 是一种基于 YAML 的简单声明式 *data-oriented* 语言, 将提示置于首要位置。PDL 与许多 LLM 平台和 LLMs 兼容良好。它支持编写调用 LLMs 和工具的交互式应用程序, 并使得实现聊天机器人、RAG 或智能体等常见用例变得容易。我们希望 PDL 能使提示编程更简单、更稳健、更令人愉悦。

1 Introduction

大型语言模型 (LLMs) 已取得重大进展, 展现出执行广泛有用任务的能力。由于 LLMs 通过自然语言 *prompts* 进行控制, 提示工程已成为一种提高准确性的临时方法 [30]。通过提示模式, 如上下文学习 [6]、链式调用多个 LLM[7]、检索增强生成 (RAG) [17]、工具使用 [27]、程序辅助语言模型 (PAL) [10] 以及智能体 [33], 还能解锁更多能力。不幸的是, 尽管功能强大, LLMs 仍然脆弱: 它们有时会产生幻觉, 甚至无法遵循预期的语法和类型。

提示框架 [19] 使开发者更容易使用 LLMs 及相关提示模式, 同时减轻其脆弱性。其中一些框架, 如 LangChain[7] 和 AutoGen[31], 通过为 RAG 或智能体等流行模式提供定制功能来实现这一点。不幸的是, 这种定制性剥夺了用户对基本提示的控制, 迫使他们学习许多复杂的框架特性。相比之下, 低层级的提示框架, 如 Guidance[22] 和 LMQL[5], 通过语法和类型提供了更多控制。然而, 它们要求用户使用 Python 或 TypeScript 等命令式语言进行编程。在另一极端, 如 DSPy[14] 和 Vieira[18] 等框架通过自动生成提示, 完全避免了手动编写提示。不幸的是, 这进一步剥夺了开发者的控制权。因此, 问题在于如何在保持简单性并让开发者掌握主导权的同时, 使 LLM 编程不那么脆弱。

为解决这一问题, 我们转向了久经考验的编程语言设计理念。*orthogonality* 原则倡导一组简单的特性, 以强大的方式组合 [29]。正交性, 或成直角关系, 在此意味着无冗余并尽可能避免特殊情况。对于提示框架而言, 正交性是避免定制功能的一种方式。其次, 如果语言检查 *types* 和 *roles*[12] 以通过构造强制执行结构, 开发者就能减少与脆弱性的斗争。一个更难以解决的矛盾依然存在: 一方面, 我们希望给予开发者对精确提示的控制权; 另一方面, 我们想要一种简单的 *declarative* 语言。为此, 我们确定了一种 *data-oriented language*, 它通过有意模糊程序 (例如用于链式调用和工具) 与数据 (用于

*IBM 研究院

†加州大学戴维斯分校

This paper introduces the Prompt Declaration Language (PDL), an orthogonal and typed data-oriented language. Unlike other prompting languages that are embedded in imperative languages, PDL is based on YAML [4]. YAML is a data serialization format that is both human-readable (by promoting a nice and simple syntax for unstructured strings) while also providing structure (by being JSON-compatible). Variables in PDL also hold JSON values and can optionally be typed with JSON Schema [24]. PDL is currently implemented by an interpreter, and the interpreter performs dynamic type checking. One benefit of representing programs as data is that it facilitates program transformations [21], such as for optimization. Rendering programs in a data representation format even facilitates PDL programs that generate PDL programs with LLMs, similar to PAL [10].

PDL programs comprise blocks (YAML objects), where each block appends data to the prompt context. This mental model is a natural fit for prompting techniques such as chatbots or agents: program execution implicitly builds up a conversation or trajectory, without necessitating explicit plumbing. This context then becomes the input to the next LLM call. PDL supports local LLMs, and LLMs hosted by various providers, including but not limited to open-source Granite models [1, 11] on IBM watsonx¹ and on Replicate². PDL provides control constructs for looping and conditionals, as well as functions and file includes for modularity. PDL adopts Jinja2 [25] expressions to template not just prompts but entire programs.

This paper gives a quick overview of PDL by means of an introductory example (Section 2), followed by a tour of the language (Section 3). It describes the tooling for running and editing PDL programs (Section 4) and offers case studies illustrating more uses of PDL (Section 5). Finally, the paper discusses related work (Section 6) and concludes (Section 7). PDL is open-source and available at <https://github.com/IBM/prompt-declaration-language>. Overall, PDL is a simple yet powerful new language for LLM prompt programming.

2 Overview

This section gives an overview of PDL features by means of a chatbot example. A PDL program executes a sequence of *blocks*, each of which generates data that it contributes to the background context. There are different kinds of blocks, capable of generating data in different ways: model calls, reading data from stdin or a file, directly creating various kinds of JSON data, and executing code. In addition, there are a variety of control blocks (if-then-else, for, and repeat) that let PDL users express rich data pipelines and AI applications.

Fig. 1(a) shows the PDL code for a simple chatbot. The **read**: block on Lines 1–4 prints a message asking the user to enter a query, which it reads from stdin. Fig. 1(b) shows an execution trace of the same program. For instance, the user might ask ‘What’s a language salad?’. To avoid duplication, the ‘**contribute**: [context]’ clause puts the user response into the background context but not the result (printed on stdout).

The **repeat:until**: block on Lines 5–16 has one nested **text**: block, which in turn has a sequence of two nested blocks. The **text**: block turns the results of its nested blocks into strings and concatenates them. The **model**: block on Lines 7–9 calls an LLM, using the accumulated context so far as the prompt. In the first loop iteration, that context comprises only two lines ‘What is your query?’ and ‘What’s a language salad?’. The ‘**stop**: [\n\n]’ model parameter causes the LLM to stop producing tokens after generating two consecutive newline characters. The LLM interpreter prints LLM outputs in green; Fig. 1(b) shows that in this example, the LLM produced ‘A language salad is [...]’. The **read**: block on Lines 10–15 prints a message using YAML’s multi-line string syntax starting with a vertical bar (|). This example illustrates how PDL keeps prompts at the forefront

¹<https://www.ibm.com/watsonx>

²<https://replicate.com/>

提示)之间的界限,将提示置于首要位置。这灵感来源于代码即数据的古老理念[20],以及关于无层级编程的开创性工作[8]。本文介绍了提示声明语言(PDL),这是一种正交且具备类型的数据导向语言。与其他嵌入在命令式语言中的提示语言不同,PDL基于YAML[4]。YAML是一种数据序列化格式,既具备人类可读性(通过为无结构字符串提供简洁优雅的语法),同时也提供结构(通过兼容JSON)。PDL中的变量PDL目前由一个解释器实现,该解释器执行动态类型检查。将程序表示为数据的一个好处是便于程序变换[21],例如用于优化。以数据表示格式渲染程序甚至促进了使用LLM生成PDL程序的PDL程序,类似于PAL[10]。

PDL程序由块(YAML对象)组成,每个块向提示上下文追加数据。这种心智模型自然契合聊天机器人或智能体等提示技术:程序执行隐式地构建对话或轨迹,无需显式的管道连接。该上下文随后成为下一次LLM调用的输入。PDL支持本地LLM以及由不同服务商托管的LLM,包括但不限于IBM watsonx¹和Replicate²上的开源Granite模型[1, 11]。PDL提供了用于循环和条件判断的控制结构,以及用于模块化的函数和文件包含功能。PDL采用Jinja2[25]表达式来模板化不仅提示,还包括整个程序。

本文通过一个介绍性示例(第2节)快速概述PDL,随后进行语言导览(第3节)。文中描述了运行和编辑PDL程序的工具(第4节),并提供了案例研究以说明PDL的更多用途(第5节)。最后,本文讨论了相关工作(第6节)并给出结论(第7节)。PDL是开源的,可在<https://github.com/IBM/prompt-declaration-language>获取。总体而言,PDL是一种简单而强大的新型LLM提示编程语言。

2 Overview

本节通过一个聊天机器人示例概述PDL的特性。PDL程序执行一系列*blocks*,每个块生成数据并将其贡献至背景上下文中。存在多种不同类型的块,能够以不同方式生成数据:模型调用、从标准输入或文件读取数据、直接创建各类JSON数据以及执行代码。此外,还有多种控制块(if-then-else、for和repeat),使PDL用户能够表达丰富的数据流水线和AI应用。

图1(a)展示了一个简单聊天机器人的PDL代码。第1–4行的**read**:块打印一条消息,提示用户输入查询,并从标准输入读取该查询。图1(b)展示了同一程序的执行轨迹。例如,用户可能询问“什么是语言沙拉?”。为避免重复,‘**contribute**: [context]’子句将用户响应放入背景上下文,但不放入结果(打印至标准输出)。

第5–16行的**repeat:until**:块包含一个嵌套的**text**:块,该块又包含两个嵌套块的序列。**text**:块将其嵌套块的结果转换为字符串并连接它们。第7–9行的**model**:块调用一个LLM,使用迄今为止累积的上下文作为提示。在第一次循环迭代中,该上下文仅包含两行:“What

¹<https://www.ibm.com/watsonx>

²<https://replicate.com/>

```

1 - read:
2   contribute: [context]
3   message: |
4     What is your query?
5 - repeat:
6   text:
7     - model: watsonx/ibm/granite-13b-chat-v2
8     parameters:
9       stop: ["\n\n"]
10  - def: question
11    read:
12    contribute: [context]
13    message: |
14
15    Enter a query or say "quit" to exit.
16 until: ${question == "quit"}

```

(a) Code

```

What is your query?
What's a language salad?
A language salad is a term used to describe a mix of
different languages and dialects in a single conver-
sation or piece of text. It can be seen as an [...]
Enter a query or say "quit" to exit.
Say it as a poem!
In a world where many tongues are sown,
A language salad is born, in joy they're grown.
A medley of words, in harmony flow,
Swirling colors of speech, in a vibrant show.
Enter a query or say "quit" to exit.

```

(b) Interpreter trace

Fig. 1. Simple Chatbot in PDL

while making them readable and giving the developer precise control. The interpreter trace on the right shows the user entering ‘Say it as a poem!’, which Line 10 on the left uses to define variable `question` and Line 12 appends to the context. The `until` clause on Line 16 specifies a Jinja2 expression ‘`${question == "quit"}`’ as the loop termination condition. PDL embeds Jinja2 templates using ‘`${...}`’ syntax rather than ‘`{{...}}`’ because the latter interacts poorly with YAML, where curly braces are special characters.

In the second loop iteration, the context includes the effect of the first loop iteration. Hence, the second execution of the `model` block sees the output from its first execution and can paraphrase it as a poem, ‘*In a world where many tongues [...]*’ in Fig. 1(b). Finally, in this example, during the second execution of the `read` block the user entered ‘quit’, causing the loop to terminate. Now that we have seen a few common PDL blocks in action (`read`, `repeat`, `text`, and `model`), we can proceed to Section 3, which describes the remaining blocks and language features.

3 Language

PDL is a language embedded into YAML such that every PDL program is a valid YAML document following the PDL schema³. Fig. 2 is a quick reference of PDL, and this section explains it using grammar rules. A program is a block or a list of blocks where blocks are expressions or structured blocks, as expressed by the following grammar rules:

```

pdl  ::= block | [block, ..., block]
block ::= expression | structured_block

```

All grammar rules in this section use YAML’s flow-style syntax (e.g., `[block, ..., block]`). The same PDL code can also be rendered in YAML’s block-style syntax, e.g.:

```

- block
...
- block

```

Each block has a block body, with keywords indicating the block kind (e.g., `model` or `read`). There are 15 kinds of block bodies (optional fields are annotated with a question mark):

³<https://ibm.github.io/prompt-declaration-language/dist/pdl-schema.json>

```

1 - read:
2   contribute: [context]
3   message: |
4     What is your query?
5 - repeat:
6   text:
7     - model: watsonx/ibm/granite-13b-chat-v2
8     parameters:
9       stop: ["\n\n"]
10  - def: question
11    read:
12    contribute: [context]
13    message: |
14
15    Enter a query or say "quit" to exit.
16 until: ${question == "quit"}

```

(a) 代码

```

您的查询是什么？
What's a language salad?
A language salad is a term used to describe a mix
of different languages and dialects in a single con-
versation or piece of text. It can be seen as an [...]
输入查询或说 “quit” 以退出。
Say it as a poem!
In a world where many tongues are sown,
A language salad is born, in joy they're grown.
A medley of words, in harmony flow,
Swirling colors of speech, in a vibrant show.
输入查询或说 “quit” 以退出。

```

(b) 解释器执行轨迹

Fig. 1. PDL 中的简单聊天机器人

is your query?” 和 “What’s a language salad?”。‘`stop: [\n\n]`’ 模型参数使 LLM 在生成两个连续换行符后停止生成令牌。LLM 解释器以绿色打印 LLM 输出；图 1(b) 显示在此示例中，LLM 生成了 ‘*A language salad is [...]*’。第 10–15 行的 `read` 块使用以竖杠 (|) 开头的 YAML 多行字符串语法打印消息。此示例说明了 PDL 如何将提示置于前沿，同时使其可读并赋予开发者精确控制。右侧的解释器轨迹显示用户输入 “Say it as a poem!”，左侧第 10 行使用该输入定义变量 `question`，第 12 行将其追加至上下文。第 16 行的 `until` 子句指定了一个 Jinja2 表达式 ‘`${question == "quit"}`’ as the loop termination condition. PDL embeds Jinja2 templates using ‘`${...}`’ 语法而非 ‘`{{...}}`’，因为后者与 YAML 交互不佳，其中花括号是特殊字符。

在第二次循环迭代中，上下文包含了第一次循环迭代的影响。因此，第二次执行 `model` 块时能看到其第一次执行的输出，并能将其释义为诗歌，即图 1(b) 中的 ‘*In a world where many tongues [...]*’。最后，在此示例中，在第二次执行 `read` 块期间，用户输入了 “quit”，导致循环终止。既然我们已经看到了几个常见 PDL 块的实际运行（`read`、`repeat`、`text` 和 `model`），我们可以继续第 3 节，该节描述了其余块和语言特性。

LLM call with current context	Including a PDL file	Optional keywords for any block
<code>model: watsonx/ibm/granite-13b-chat-v2</code> <code>parameters:</code> <code>temperature: 0.1</code>	<code>include: ./helper_defs.pdl</code>	<code>description:</code> documentation text <code>def: x</code> # define variable from block
LLM call with explicit input	Declaring and calling functions	
<code>model: watsonx/ibm/granite-13b-chat-v2</code> <code>parameters:</code> <code>temperature: 0.1</code> <code>input:</code> <code>array:</code> <code>- role: user</code> <code>content: Hello,</code>	<code>def: add</code> <code>function:</code> <code>x: int</code> <code>y: int</code> <code>return: \$(x + y)</code> <code>call: add</code> <code>args:</code> <code>x: 2</code> <code>y: 2</code> <code>pdl_context: []</code> # optional	<code>defs:</code> # define multiple variables <code>x: v1</code> <code>y: v2</code> <code>role: user</code> # or system or assistant <code>contribute:</code> [result, context] # or less <code>parser: json</code> # or jsonl, yaml, regex <code>spec: type</code> # type specification
Reading from a file or stdin		spec Types (shorthand for JSON Schema)
<code>read:</code> # optionally, add file name <code>message:</code> Please enter an input. <code>multiline: true</code> # omit to stop at \n		Basic types <code>str, int, float, bool, null</code> Arrays <code>[int]</code> Objects <code>{x: int, y: int}</code> Enums <code>(enum: [red, green, blue])</code>
Creating data (v1, v2 can be any block)	Control constructs (all lists use implicit lastOf)	
<code>text:</code> # outputs "v1v2" <code>- v1</code> <code>- v2</code> <code>lastOf:</code> # outputs v2 <code>- v1</code> <code>- v2</code> # implicit lastOf, outputs v2 <code>- v1</code> <code>- v2</code> <code>array:</code> # outputs [v1, v2] <code>- v1</code> <code>- v2</code> <code>object:</code> # outputs {k1: v1, k2: v2} <code>k1: v1</code> <code>k2: v2</code> <code>data:</code> # outputs {k1: v1, model: v2} <code>k1: v1</code> <code>model: v2</code> # no LLM call	<code>if: \$(x > 0)</code> <code>then: positive</code> <code>else: non-negative</code> <code>for:</code> # outputs 2_0_5 <code>i: [1, 0, 1]</code> <code>j: [2, 3, 5]</code> <code>repeat: \$(i * j)</code> <code>join:</code> <code>with: _</code> # optional <code>repeat:</code> # implicit lastOf, outputs HoHoHo <code>- Hi</code> <code>- Ho</code> <code>num_iterations: 3</code> <code>repeat:</code> # outputs HiHoHiHoHiHo <code>text:</code> <code>- Hi</code> <code>- Ho</code> <code>num_iterations: 3</code> <code>repeat:</code> <code>def: x</code> <code>read:</code> <code>until: \$(x trim) == "stop"</code>	<code>Basic values</code> "hi", 5, 3.1, true, none <code>Arrays/objects</code> [1, 2, 3], {"x": 4, "y": 5} <code>Variables</code> x, y[0], z.f <code>Operators</code> +, -, *, /, %, **, ~, and, or, not, ==, <, >, in <code>Tests</code> x if x is defined else 0 <code>Filters</code> x default(0)
		Executing code
		<code>lang: python</code> <code>code:</code> <code>result = "Hello, world!"</code>

Fig. 2. PDL Quick Reference

```

block_body ::= model:expression,input:?pdl,parameters:?expression
| read:file,message:?string,message:?bool
| text:pdl
| lastOf:pdl
| array:pdl
| object:pdl
| data:json
| include:file
| function:args,return:pdl
| call:f,args:args
| if:expression,then:pdl,else:?pdl
| for:args,repeat:pdl,join:?join
| repeat:pdl,num_iterations:n,join:?join
| repeat:pdl,until:expression,join:?join
| code:pdl,lang:string

```

We already saw `model:` and `read:` blocks in the previous section. A `model:` block calls an LLM. The prompt comes from the current context, unless the optional `input:` field is specified, in which case it comes from there. The optional `parameters:` configure the model inferencing behavior. A `read:` block reads input from a file or from stdin if no file name is specified. The optional `message:` is displayed to the user and the optional `multiline:` field determines whether to stop at newline.

There are five kinds of blocks for creating data: `text:`, `lastOf:`, `array:`, `object:`, and `data:`. Fig. 2 illustrates them on simple examples. A plain list of blocks without a keyword behaves like `lastOf:`.

LLM call with current context	Including a PDL file	Optional keywords for any block
<code>model: watsonx/ibm/granite-13b-chat-v2</code> <code>parameters:</code> <code>temperature: 0.1</code>	<code>include: ./helper_defs.pdl</code>	<code>description:</code> documentation text <code>def: x</code> # define variable from block
LLM call with explicit input	Declaring and calling functions	
<code>model: watsonx/ibm/granite-13b-chat-v2</code> <code>parameters:</code> <code>temperature: 0.1</code> <code>input:</code> <code>array:</code> <code>- role: user</code> <code>content: Hello,</code>	<code>def: add</code> <code>function:</code> <code>x: int</code> <code>y: int</code> <code>return: \$(x + y)</code> <code>call: add</code> <code>args:</code> <code>x: 2</code> <code>y: 2</code> <code>pdl_context: []</code> # optional	<code>defs:</code> # define multiple variables <code>x: v1</code> <code>y: v2</code> <code>role: user</code> # or system or assistant <code>contribute:</code> [result, context] # or less <code>parser: json</code> # or jsonl, yaml, regex <code>spec: type</code> # type specification
Reading from a file or stdin		spec Types (shorthand for JSON Schema)
<code>read:</code> # optionally, add file name <code>message:</code> Please enter an input. <code>multiline: true</code> # omit to stop at \n		Basic types <code>str, int, float, bool, null</code> Arrays <code>[int]</code> Objects <code>{x: int, y: int}</code> Enums <code>(enum: [red, green, blue])</code>
Creating data (v1, v2 can be any block)	Control constructs (all lists use implicit lastOf)	
<code>text:</code> # outputs "v1v2" <code>- v1</code> <code>- v2</code> <code>lastOf:</code> # outputs v2 <code>- v1</code> <code>- v2</code> # implicit lastOf, outputs v2 <code>- v1</code> <code>- v2</code> <code>array:</code> # outputs [v1, v2] <code>- v1</code> <code>- v2</code> <code>object:</code> # outputs {k1: v1, k2: v2} <code>k1: v1</code> <code>k2: v2</code> <code>data:</code> # outputs {k1: v1, model: v2} <code>k1: v1</code> <code>model: v2</code> # no LLM call	<code>if: \$(x > 0)</code> <code>then: positive</code> <code>else: non-negative</code> <code>for:</code> # outputs 2_0_5 <code>i: [1, 0, 1]</code> <code>j: [2, 3, 5]</code> <code>repeat: \$(i * j)</code> <code>join:</code> <code>with: _</code> # optional <code>repeat:</code> # implicit lastOf, outputs HoHoHo <code>- Hi</code> <code>- Ho</code> <code>num_iterations: 3</code> <code>repeat:</code> # outputs HiHoHiHoHiHo <code>text:</code> <code>- Hi</code> <code>- Ho</code> <code>num_iterations: 3</code> <code>repeat:</code> <code>def: x</code> <code>read:</code> <code>until: \$(x trim) == "stop"</code>	<code>Basic values</code> "hi", 5, 3.1, true, none <code>Arrays/objects</code> [1, 2, 3], {"x": 4, "y": 5} <code>Variables</code> x, y[0], z.f <code>Operators</code> +, -, *, /, %, **, ~, and, or, not, ==, <, >, in <code>Tests</code> x if x is defined else 0 <code>Filters</code> x default(0)
		Executing code
		<code>lang: python</code> <code>code:</code> <code>result = "Hello, world!"</code>

Fig. 2. PDL 快速参考

3 Language

PDL 是一种嵌入到 YAML 中的语言，因此每个 PDL 程序都是一个遵循 PDL 模式³的有效 YAML 文档。图 2 是 PDL 的快速参考，本节使用语法规则对其进行解释。一个程序是一个块或块的列表，其中块是表达式或结构化块，如下列语法规则所示：

```

pdl ::= block | [block, ..., block]
block ::= expression | structured_block

```

本节中的所有语法规则都使用 YAML 的流式语法（例如，`[block, ..., block]`）。相同的 PDL 代码也可以用 YAML 的块式语法呈现，例如：

```

- block
...
- block

```

每个块都有一个块体，其中的关键字指示块的种类（例如，`model` 或 `read`）。共有 15 种块体（可选字段用问号标注）：

³<https://ibm.github.io/prompt-declaration-language/dist/pdl-schema.json>

The difference between an **object**: block and a **data**: block is that the PDL interpreter ignores PDL keywords in the latter, treating them like plain JSON fields instead.

For modularity, PDL supports **include**: blocks and functions. An **include**: block opens the PDL program at the given relative path and adds its output at the place where it occurred. Function arguments have the following syntax:

```
args ::= {x: expression, ..., x: expression}
```

Each x : *expression* maps an argument name to either a type specification (in a **function**: definition) or a value (in a function call:). The **return**: keyword provides the function body, which can have nested blocks; Fig. 2 shows a simple case where it is just a Jinja2 expression. The optional **pdl_context**: keyword can reset the background context for the duration of a call, e.g. to the empty context [].

There are three kinds of blocks for control constructs: **if**:, **for**:, and different flavors of **repeat**:. They can have nested blocks or simple expressions, and if they contain a list of blocks, that list implicitly behaves like **lastOf**:. If the **lastOf**: behavior is not intended, a common idiom is to wrap the body of a loop in a **text**: block, or to combine loop iteration results with the **join**: keyword:

```
join ::= as:?(text | array | lastOf),with:?'string'
```

Each of the 15 kinds of block bodies described above can be orthogonally composed with zero or more optional keywords that work for any block:

```
structured_block ::= { block_body,
  description:?'string',
  def:?'x',
  defs:?'defs',
  role:?'string',
  contribute:?'contribute',
  parser:?'parser',
  spec:?'type' }
```

A **description**: is a special comment. A **def**: assigns the result of the block to a variable; we already saw an example for that in Line 10 of Fig. 1. In contrast, **defs**: creates multiple variable definitions, each with its own name x and a value given by a nested PDL program:

```
defs ::= {x:pdl, ..., x:pdl}
```

A **role**: causes the data resulting from a block to be decorated with a role, such as 'user', 'assistant', or 'system'. When PDL calls a chat model, it follows common practice of modern chat APIs and passes not a flat string as the prompt, but rather, a sequence of {content:str, role:str} pairs. Then, the model API applies a model-specific chat template, which flattens that sequence by inserting the appropriate control tokens for that model [12]. This gives PDL programs some degree of model-independence. If a block does not have an explicit **role**:, it defaults to 'assistant' for model blocks and to 'user' for all other blocks. Inner nested blocks have the same role as their outer enclosing block. In future work, we also plan to leverage roles for privilege-based security.

The **contribute**: keyword can specify a (possibly empty) subset of the two destinations 'result' or 'context'. By default, every block contributes to both its own result and the background context for later LLM calls. Line 2 of Fig. 1 showed an example of limiting the contribution of a block to just the context to declutter the output.

The **parser**: keyword makes it possible for a block that would ordinarily just produce a flat string (e.g., an LLM call) to instead produce structured data. The supported parsers are json, yaml, regex, and jsonl. The **spec**: keyword can specify a type. PDL's types are a subset of JSON Schema [24], with shorthand syntax for simple commonly used types illustrated in Fig. 2. For example, type '{questions: [str], answers: [str]}' is an object with two fields questions and answers, both of which

```
block_body ::= model:expression, input:?'pdl', parameters:?'expression'
| read:file, message:?'string', message:?'bool'
| text:pdl
| lastOf:pdl
| array:pdl
| object:pdl
| data:json
| include:file
| function:args, return:pdl
| call:f, args:args
| if:expression, then:pdl, else:?'pdl'
| for:args, repeat:pdl, join:?'join'
| repeat:pdl, num_iterations:n, join:?'join'
| repeat:pdl, until:expression, join:?'join'
| code:pdl, lang:string
```

我们已经在上一节中看到了 **model**: 和 **read**: 块。一个 **model**: 块会调用一个 LLM。提示词来自当前上下文，除非指定了可选的 **input**: 字段，则提示词来自该字段。可选的 **parameters**: 用于配置模型推理行为。一个 **read**: 块从文件读取输入，如果未指定文件名，则从 stdin 读取。可选的 **message**: 会显示给用户，可选的 **multiline**: 字段决定是否在新行处停止。

有五种用于创建数据的块: **text**:、**lastOf**:、**array**:、**object**: 和 **data**:。图 2 通过简单示例对它们进行了说明。一个没有关键字的普通块列表的行为类似于 **lastOf**:。 **object**: 块与 **data**: 块之间的区别在于，PDL 解释器会忽略后者中的 PDL 关键字，将其视为普通的 JSON 字段。为了模块化，PDL 支持 **include**: 块和函数。一个 **include**: 块会打开给定相对路径处的 PDL 程序，并将其输出添加到它出现的位置。函数参数具有以下语法:

```
args ::= {x: expression, ..., x: expression}
```

每个 x : *expression* 将一个参数名映射到一个类型说明（在 **function**: 定义中）或一个值（在函数 call: 中）。 **return**: 关键字提供了函数体，它可以包含嵌套块；图 2 展示了一个简单情况，其中它只是一个 Jinja2 表达式。可选的 **pdl_context**: 关键字可以在调用期间重置背景上下文，例如重置为空上下文 []。

控制结构有三种类型的块: **if**:、**for**: 和不同变体的 **repeat**:。它们可以包含嵌套块或简单表达式，并且如果它们包含一个块列表，该列表会隐式地表现得像 **lastOf**:。如果不希望出现 **lastOf**: 行为，一种常见的做法是将循环体包装在一个 **text**: 块中，或者使用 **join**: 关键字组合循环迭代结果:

```
join ::= as:?(text | array | lastOf),with:?'string'
```

上述描述的 15 种块体中的每一种都可以正交地与零个或多个适用于任何块的可选关键字组合:

hold arrays of strings. Section 5 will illustrate how `parser:` and `spec:` can work together. Future work will also leverage these keywords for constrained decoding [28].

An atomic block is an expression:

```
expression ::= bool | number | string | ${jinja_expression} | string_expression
```

Expressions can be basic values, Jinja2 expressions [25], or strings containing Jinja expressions. Jinja2 is a convenient way for specifying prompt templates, where parts of a prompt are hardcoded and others are filled in from expressions. But PDL takes the use of Jinja2 further, by letting developers template not just individual prompts, but entire chains of model calls and other blocks. While we refer the reader to the Jinja2 documentation for an exhaustive list of possible expressions, Fig. 2 briefly lists the most common ones. PDL adopts only Jinja2 expressions, not Jinja2 statements such as `{% if .. %}` or `{% for .. %}`, because those are redundant with PDL's own `if:` and `for:`.

Last but not least, PDL has a `code:` block that allows it to execute code in a given programming language (at the time of writing, only Python is supported). The next section will describe PDL tooling, including the interpreter, which provides a sandboxing feature to reduce risks associated with executing arbitrary code. To learn more, see the tutorial linked from PDL's github repository.

4 Tooling

PDL comes with tools for making PDL programs easy to write, run, and understand.

First and foremost, the PDL *interpreter* is an execution engine with a command-line interface, as one would expect from a scripting language. The interpreter supports a streaming mode, where LLM outputs become visible incrementally as they are being produced, for a more interactive chat experience. The interpreter also supports sandboxing, which causes it to launch in a container, recommended when executing LLM-generated actions or code.

The PDL *IDE support* enhances VSCode, making it easier to write PDL code via syntax highlighting, auto-complete, tooltips for PDL keywords, and error checking. These capabilities are, in part, driven by the PDL meta-schema i.e., the JSON schema that defines what constitutes valid PDL.

The `%pdl cell magic` enhances Jupyter Notebooks so developers can write code cells directly in PDL. That way, hosted notebook platforms can serve as a simple playground for interactively exploring prompts. Given multiple PDL code cells in the same notebook, later cells can use variables defined in earlier cells. Furthermore, the background context for later cells is continued from earlier cells; when not desired, developers can override this behavior via `%pdl --reset-context`.

The PDL *live document visualizer* shows the concrete execution trace of a PDL program with colored nested boxes, similar to typical figures in papers or blog posts about LLM prompting. Then, the user can select one of the boxes to display the corresponding PDL code, similar to how spreadsheet cells show data, but the user can select them to inspect the formula that created that data. This live view is a way to let users quickly understand concrete data, and then move from that to understanding the code that produced it.

Finally, PDL has an *SDK* (software development kit), which is a small Python library for calling from Python into PDL. This is useful for extending larger Python applications to use prompt-based programs, such as agents. As discussed in Section 3, a PDL file can contain Python in `code:` blocks. When developing larger applications with PDL, we have found it useful to keep these to a few lines of code, by defining a function in a separate Python file and then calling it from PDL. A good practice is to pass data from PDL to Python and vice versa as JSON objects. Optionally, this can be type-checked using the `spec:` keyword in PDL, and TypedDict or Pydantic on the Python side, as illustrated in the next section in Fig. 3.

```
structured_block ::= { block_body,
                        description: ?string,
                        def: ?x,
                        defs: ?defs,
                        role: ?string,
                        contribute: ?contribute,
                        parser: ?parser,
                        spec: ?type }
```

description: 是一种特殊的注释。**def:** 将块的结果赋值给一个变量；我们已经在图 1 的第 10 行看到了一个例子。相比之下，**defs:** 创建多个变量定义，每个定义都有自己的名称 `x` 和一个由嵌套 PDL 程序给出的值：

```
defs ::= {x: pdl, ..., x: pdl}
```

role: 会导致块产生的数据被装饰一个角色，例如 ‘user’、‘assistant’ 或 ‘system’。当 PDL 调用聊天模型时，它遵循现代聊天 API 的常见做法，传递的不是一个扁平字符串作为提示，而是一个由 `{content:str, role:str}` 对组成的序列。然后，模型 API 应用一个模型特定的聊天模板，该模板通过插入该模型适当的控制令牌来扁平化该序列 [12]。这使 PDL 程序具有一定程度的模型无关性。如果一个块没有显式的 **role:**，则对于模型块默认为 ‘assistant’，对于所有其他块默认为 ‘user’。内部嵌套块具有与其外部封闭块相同的角色。在未来的工作中，我们还计划利用角色实现基于权限的安全性。**contribute:** 关键字可以指定一个（可能为空的）目标子集，即 ‘result’ 或 ‘context’。默认情况下，每个块都会贡献到其自身的结果以及后续 LLM 调用的背景上下文中。图 1 的第 2 行展示了一个将块的贡献限制为仅上下文的例子，以简化输出。

parser: 关键字使得一个通常只产生扁平字符串（例如，一个 LLM 调用）的块能够转而产生结构化数据。支持的解析器有 json、yaml、regex 和 jsonl。**spec:** 关键字可以指定一个类型。PDL 的类型是 JSON Schema [24] 的一个子集，其简单常用类型的简写语法如图 2 所示。例如，类型 `{questions: [str], answers: [str]}` 是一个具有两个字段 `questions` 和 `answers` 的对象，这两个字段都包含字符串数组。第 5 节将说明 **parser:** 和 **spec:** 如何协同工作。未来的工作也将利用这些关键字进行约束解码 [28]。

一个原子块是一个表达式：

```
expression ::= bool | number | string | ${jinja_expression} | string_expression
```

表达式可以是基本值、Jinja2 表达式 [25]，或包含 Jinja 表达式的字符串。Jinja2 是指定提示模板的一种便捷方式，其中部分提示是硬编码的，其他部分则通过表达式填充。但 PDL 进一步扩展了 Jinja2 的用途，允许开发者不仅模板化单个提示，还可以模板化整个模型调用链及其他块。虽然我们建议读者查阅 Jinja2 文档以获取可能表达式的详尽列表，但图 2 简要列出了最常见的表达式。PDL 仅采用 Jinja2 表达式，而不采用 Jinja2 语

```

1 text:
2 - lang: python
3 code: |
4     import rag_mbpp
5     PDL_SESSION.mbpp = rag_mbpp.initialize()
6     result = ""
7 - defs:
8     test_query: >-
9         Write a python function to remove first and
10        last occurrence of a given character from
11        the string.
12 retrieved:
13     lang: python
14     spec: [[query: str, answer: str]]
15     code: |
16         import rag_mbpp
17         result = rag_mbpp.retrieve(
18             PDL_SESSION.mbpp, "${test_query}", 5
19         )
20 text: >
21     Given the text after "Q:", generate a Python
22     function after "A:".
23
24     Here are some examples, complete the last one:
25 - for:
26     few_shot_sample: ${retrieved}
27 repeat: |
28     Q: ${few_shot_sample.query}
29     A: ~~~${few_shot_sample.answer}~~~
30 - | -
31     Q: ${test_query}
32     A:
33 - model: watsonx/ibm/granite-3-8b-instruct
34 parameters:
35     stop: ["Q:", "A:"]

```

(a) PDL code

```

1 from typing import TypedDict
2 import datasets
3 from sklearn.feature_extraction.text \
4     import TfidfVectorizer
5
6 def initialize():
7     train_in = datasets.load_dataset(
8         "mbpp", "sanitized", split="train"
9     )
10    corpus = [row["prompt"] for row in train_in]
11    tfidf = TfidfVectorizer().fit(corpus)
12    def embed(text):
13        sparse_result = tfidf.transform(
14            raw_documents=[text]
15        )
16        return sparse_result.toarray().flatten()
17    train_em = train_in.map(
18        lambda row: {"em": embed(row["prompt"])})
19    )
20    vec_db = train_em.add_faiss_index("em")
21    return vec_db, embed
22
23 QA = TypedDict("QA", {"query": str, "answer": str})
24 def retrieve(mbpp, query, n: int) -> list[QA]:
25     vec_db, embed = mbpp
26     key = embed(query)
27     nearest = vec_db.get_nearest_examples(
28         "em", key, n
29     )
30     queries = nearest.examples["prompt"]
31     answers = nearest.examples["code"]
32     return [
33         {"query": q, "answer": a}
34         for q, a in zip(queries, answers)
35     ]

```

(b) Python code

Fig. 3. RAG example in PDL

5 Case Studies

We already saw a simple PDL chatbot example in Section 2. This section illustrates slightly more advanced use-cases for PDL: RAG, agents, and generating PDL from PDL.

5.1 Retrieval-Augmented Generation

Retrieval-augmented generation, or RAG, works by first retrieving relevant context, then adding that to the prompt for a model to generate an answer [17]. Fig. 3(a) shows a PDL program that uses RAG to retrieve few-shot samples for a code-generation task. The **code:** block in Lines 2–6 uses Python to initialize a vector database for the training split of the MBPP dataset of “mostly basic Python programs” [2]. It uses a Python function defined in Fig. 3(b), together with a PDL_SESSION special variable that enables it to carry state to a later code block. Lines 8–11 of Fig. 3(a) initialize variable **test_query** with a natural-language request for Python code to be generated. Lines 12–19 initialize variable **retrieved** with the five most similar samples from the training data.

句，例如{% if .. %} or {% for .. %}, because 因为这些与 PDL 自身的 **if:** 和 **for:** 是冗余的。

最后但同样重要的是，PDL 有一个 **code:** 块，允许它在给定的编程语言中执行代码（在撰写本文时，仅支持 Python）。下一节将描述 PDL 工具，包括解释器，它提供了沙箱功能以减少执行任意代码相关的风险。要了解更多信息，请参阅 PDL 的 GitHub 仓库中链接的教程。

4 Tooling

PDL 附带了一系列工具，使得编写、运行和理解 PDL 程序变得容易。

首先，PDL *interpreter* 是一个具有命令行界面的执行引擎，正如人们对脚本语言的期望。该解释器支持流式模式，在这种模式下，LLM 的输出会在生成过程中逐步显示，从而提供更具交互性的聊天体验。该解释器还支持沙盒功能，这会导致它在容器中启动，建议在执行 LLM 生成的操作或代码时使用。

PDL *IDE support* 增强了 VSCode，通过语法高亮、自动补全、PDL 关键字的工具提示和错误检查，使得编写 PDL 代码更加容易。这些功能部分是由 PDL 元模式驱动的，即定义有效 PDL 构成的 JSON 模式。

%pdl *cell magic* enhances Jupyter Notebooks 因此，开发者可以直接在 PDL 中编写代码单元。这样，托管的笔记本平台就可以作为一个简单的游乐场，用于交互式地探索提示。给定同一笔记本中的多个 PDL 代码单元，后面的单元可以使用前面单元中定义的变量。此外，后面单元的背景上下文会从前面的单元延续；当不需要此行为时，开发者可以通过 %pdl --reset-context。

PDL *live document visualizer* 以彩色嵌套框的形式显示 PDL 程序的具体执行轨迹，类似于关于 LLM 提示的论文或博客文章中常见的图示。然后，用户可以选择其中一个框来显示相应的 PDL 代码，类似于电子表格单元格显示数据的方式，但用户可以选择它们来检查生成该数据的公式。这种实时视图是一种让用户快速理解具体数据，然后从数据转向理解生成它的代码的方式。

最后，PDL 有一个 SDK（软件开发工具包），这是一个用于从 Python 调用 PDL 的小型 Python 库。这对于扩展更大的 Python 应用程序以使用基于提示的程序（例如智能体）非常有用。如第 3 节所述，PDL 文件可以在 **code:** 块中包含 Python 代码。在使用 PDL 开发大型应用程序时，我们发现通过在一个单独的 Python 文件中定义一个函数，然后从 PDL 中调用它，将这些代码保持在几行之内是很有用的。一个好的做法是将数据以 JSON 对象的形式从 PDL 传递到 Python，反之亦然。可选地，这可以使用 PDL 中的 **spec:** 关键字以及 Python 端的 TypedDict 或 Pydantic 进行类型检查，如图 3 的下一节所示。

```

1 text:
2 - read: react_few_shot_samples.txt
3 - |
4
5 When was the discoverer of the Hudson River born?
6 - repeat:
7   text:
8   - def: thought
9     model: watsonx/ibm/granite-34b-code-instruct
10    parameters:
11      stop: ["Act:"]
12      include_stop_sequence: true
13   - def: action
14     model: watsonx/ibm/granite-34b-code-instruct
15    parameters:
16      stop: ["\n"]
17    parser: json
18    spec: {name: str, arguments: {topic: str}}
19   - def: observation
20     if: ${ action.name == "Search" }
21     then:
22       text:
23       - "Obs: "
24       - lang: python
25       code: |
26         import wikipedia
27         query = "${ action.arguments.topic }"
28         result = wikipedia.summary(query)
29 until: ${ action.name != "Search" }

```

(a) Code

```

What is the elevation range for the area that the
eastern sector of the Colorado orogeny extends
into?
Tho: I need to search Colorado orogeny, find the
area that the eastern sector of the Colorado [...]
Act: {"name": "Search", "arguments": {"topic":
"Colorado orogeny"}}
Obs: The Colorado orogeny was an episode [...]
[...]

When was the discoverer of the Hudson River
born?
Tho: I need to search the discoverer of the Hud-
son River, find when he was born.
Act: {"name": "Search", "arguments": {"topic": "dis-
coverer of the Hudson River"}}
Obs: The Hudson River is a 315-mile [...]
Tho: The discoverer of the Hudson River is Henry
Hudson. I need to search Henry Hudson, find
when he was born.
Act: {"name": "Search", "arguments": {"topic":
"Henry Hudson"}}
Obs: Henry Hudson (c. 1565 – disappeared [...])
Tho: Henry Hudson was born in 1565. Act:
{"name": "Finish", "arguments": {"topic": "1565"}}

```

(b) Interpreter trace

Fig. 4. ReAct agent in PDL

Lines 20–24 add an instruction to the context, Lines 25–29 add the few-shot samples to the context, and Lines 30–32 add the test query to the context. The **for:** loop on Line 25 is an idiomatic way to use PDL for generating data, in this case, for in-context learning. Finally, Lines 33–35 call a Granite 3 model [11] with the accumulated context, causing it to generate the Python function requested by the test query. While this is a simple example, we have also used PDL with Codellm-Devkit [15], which performs static analysis on source code from various programming languages to retrieve other relevant context when prompting LLMs for coding tasks.

5.2 ReAct Agent

An LLM-based *agent* lets an LLM select and configure *actions*, executes those actions in an *environment*, and feeds the outputs from the actions back to the LLM as *observations*. There are different patterns for such agents, such as ReAct [33] and ReWOO [32]. An action is an LLM-based *tool call* [27], and an agent chains together multiple tool calls in a dynamic LLM-directed sequence. The ambition is to make AI-based applications less prescriptive and more goal-driven. Moreover, when something goes wrong with an action, agents can use the observation as feedback to recover.

Fig. 4 shows a PDL example of a simple ReAct agent. The core of ReAct is a think-act-observe loop, which manifests in the code as variable definitions for *thought* (Line 8), *action* (Line 13), and *observation* (Line 19). The *thought* is model-generated natural language e.g., ‘I need to search the discoverer of the Hudson River, find when he was born’ in the interpreter trace in Fig. 4(b). The

5 Case Studies

我们已在第 2 节中见过一个简单的 PDL 聊天机器人示例。本节将阐述 PDL 在稍复杂场景下的应用：RAG、智能体以及从 PDL 生成 PDL。

5.1 Retrieval-Augmented Generation

检索增强生成，或称 *RAG*，其工作原理是首先检索相关上下文，然后将其添加到提示中，供模型生成答案 [17]。图 3 (a) 展示了一个使用 RAG 为代码生成任务检索少样本示例的 PDL 程序。第 2-6 行的 **code:** 代码块使用 Python 为 MBPP 数据集（“主要是基础 Python 程序”数据集）的训练分割初始化了一个向量数据库 [2]。它使用了图 3 (b) 中定义的 Python 函数，以及一个 `PDL_SESSION` 特殊变量，该变量使其能够将状态传递到后续的代码块。图 3 (a) 的第 8-11 行用自然语言请求生成 Python 代码来初始化变量 `test_query`。第 12-19 行用训练数据中五个最相似的样本来初始化变量 `retrieved`。

第 20-24 行向上下文中添加了一条指令，第 25-29 行向上下文中添加了少样本示例，而第 30-32 行则将测试查询添加到上下文中。第 25 行的 **for:** 循环是使用 PDL 生成数据的一种惯用方式，在本例中用于上下文学习。最后，第 33-35 行使用累积的上下文调用 Granite 3 模型 [11]，使其生成测试查询所请求的 Python 函数。虽然这是一个简单的示例，但我们也已将 PDL 与 Codellm-Devkit [15] 结合使用，后者对来自各种编程语言的源代码执行静态分析，以便在提示 LLM 执行编码任务时检索其他相关上下文。

5.2 ReAct Agent

基于大语言模型（LLM）的 *agent* 让 LLM 能够选择并配置 *actions*，在 *environment* 中执行这些动作，并将动作的输出作为 *observations* 反馈给 LLM。此类智能体存在不同的模式，例如 ReAct [33] 和 ReWOO [32]。一个动作是一个基于 LLM 的 *tool call* [27]，而智能体则将多个工具调用以 LLM 引导的动态序列链接在一起。其目标是使基于 AI 的应用程序减少预设性，更具目标驱动性。此外，当某个动作出现问题时，智能体可以利用观察结果作为反馈进行恢复。

图 4 展示了一个简单 ReAct 智能体的 PDL 示例。ReAct 的核心是一个思考-行动-观察循环，这在代码中体现为 *thought*（第 8 行）、*action*（第 13 行）和 *observation*（第 19 行）的变量定义。*thought* 是模型生成的自然语言，例如图 4(b) 解释器追踪中的 ‘I need to search the discoverer of the Hudson River, find when he was born’。*action* 是模型生成的 JSON，以匹配 Granite 模型 [1] 的工具使用训练数据。左侧的第 17 行和第 18 行确保 LLM 输出被解析为 JSON 并符合 {name, arguments} 模式，右侧的解释器追踪显示模型确实生成了这样一个对象。这使得可以通过 Jinja2 访问对象的字段，例如第 27 行的 `${ action.arguments.topic }`。*observation* 由环境生成，在本例中是调用 Wikipedia 的 Python 代码。如第 4 节所讨论的，对于此类涉及运行（部分）由 LLM 生成的代码的情况，我们推荐使用 PDL 的沙箱功能。

`action` is model-generated JSON to match the tool-use training data of Granite models [1]. Lines 17 and 18 on the left ensure that the LLM output is parsed as JSON and conforms to a `{name, arguments}` schema, and the interpreter trace on the right shows that the model indeed generates such an object. That makes it possible to access fields of the object with Jinja2, such as `{{ action.arguments.topic }}` on Line 27. The observation is generated by the environment, in this case, Python code calling Wikipedia. As discussed in Section 4, for cases like this that involve running code (partially) generated by an LLM, we recommended PDL's sandboxing feature.

The interpreter trace in Fig. 4(b) shows that this execution had two iterations of the agentic loop. While this is a simple example, we have also used PDL to implement a code editing agent that we used as part of a submission⁴ to the SWE-bench Lite leaderboard [13]. This submission was the first to resolve 23.7% of instances with only open-source models, which is higher than any previous results with open-source models, and in the same ball-park as frontier models.

5.3 Generating PDL from PDL with LLMs

The previous sections showed examples of how a human developer can use PDL to encode different prompting patterns. This section turns to LLMs and shows how they can also be used to generate PDL. This meta PDL generation is helpful when LLMs need to create a plan for solving a problem, for example as part of an agentic workflow. Traditionally, such plans are just text or JSON or Python code. With PDL, these plans can be a composition of model and code calls that are fully executable. This section explores using PDL meta generation for the GSMHard dataset⁵.

GSMHard is a harder version of GSM8k, which consists of grade-school math problems that require simple arithmetic or symbolic reasoning. GSMHard contains an input which is a math problem statement, together with an output which is Python code that solves the problem. We implemented the PAL [10] approach but instead of generating Python code, we ask an LLM to generate PDL. The textual chain-of-thought is represented as PDL text blocks, and arithmetic is done using PDL code blocks.

Fig. 5 shows a PDL program that generates PDL code and executes it all in the same program. The `demos` variable holds few-shot samples designed to teach a model how to generate PDL code. On Line 32, a model call block uses these samples, together with a `question`, which is a free variable, as input. The result is a PDL program to solve the question. Line 38 extracts the PDL program and executes it in Python. This program is applied to the GSMHard dataset, where `question` is filled with input questions.

This experiment resulted in the discovery that 10% of the GSMHard dataset is actually incorrect in the sense that the ground truth is inconsistent with the question that was asked. Fig. 6 shows an example of such an inconsistency. Using PDL helped in this discovery because the generated PDL code is human-readable, so we were able to easily check data points that did not match the ground truth and found that the ground truth is incorrect in some cases. We used an LLM to cover the entire dataset and systematically pick examples that seemed inconsistent. We then manually filtered the result to remove false positives and identified 10% of data points that present this problem.

6 Related Work

A recent survey defines a *prompting framework* as a layer that manages, simplifies and facilitates the interaction between LLMs and users, tools, or other models [19]. The survey highlights that a major pitfall of prompting frameworks is increasingly steep learning curves.

⁴https://github.com/swe-bench/experiments/tree/main/evaluation/lite/20241016_IBM-SWE-1.0

⁵<https://huggingface.co/datasets/reasoning-machines/gsm-hard>

```

1  text:
2  - lang: python
3  code: |
4      import rag_mbpp
5      PDL_SESSION.mbpp = rag_mbpp.initialize()
6      result = ""
7  - defs:
8      test_query: >-
9          Write a python function to remove first and
10         last occurrence of a given character from
11         the string.
12  retrieved:
13      lang: python
14      spec: [{query: str, answer: str}]
15      code: |
16          import rag_mbpp
17          result = rag_mbpp.retrieve(
18              PDL_SESSION.mbpp, "${test_query}", 5
19          )
20  text: >
21      Given the text after "Q:", generate a Python
22      function after "A:".
23
24      Here are some examples, complete the last one:
25  - for:
26      few_shot_sample: ${retrieved}
27  repeat: |
28      Q: ${few_shot_sample.query}
29      A: ```${few_shot_sample.answer}```
30  - |-
31      Q: ${test_query}
32      A:
33  - model: watsonx/ibm/granite-3-8b-instruct
34      parameters:
35          stop: ["Q:", "A:"]

```

```

1  from typing import TypedDict
2  import datasets
3  from sklearn.feature_extraction.text \
4      import TfidfVectorizer
5
6  def initialize():
7      train_in = datasets.load_dataset(
8          "mbpp", "sanitized", split="train"
9      )
10     corpus = [row["prompt"] for row in train_in]
11     tfidf = TfidfVectorizer().fit(corpus)
12     def embed(text):
13         sparse_result = tfidf.transform(
14             raw_documents=[text]
15         )
16         return sparse_result.toarray().flatten()
17     train_em = train_in.map(
18         lambda row: {"em": embed(row["prompt"])}
19     )
20     vec_db = train_em.add_faiss_index("em")
21     return vec_db, embed
22
23 QA = TypedDict("QA", {"query": str, "answer": str})
24 def retrieve(mbpp, query, n: int) -> list[QA]:
25     vec_db, embed = mbpp
26     key = embed(query)
27     nearest = vec_db.get_nearest_examples(
28         "em", key, n
29     )
30     queries = nearest.examples["prompt"]
31     answers = nearest.examples["code"]
32     return [
33         {"query": q, "answer": a}
34         for q, a in zip(queries, answers)
35     ]

```

(a) PDL 代码

(b) Python 代码

Fig. 3. PDL 中的 RAG 示例

图 4(b) 中的解释器追踪显示, 此执行过程经历了两次智能体循环迭代。虽然这是一个简单的例子, 但我们也使用 PDL 实现了一个代码编辑智能体, 并将其作为提交⁴的一部分, 用于 SWE-bench Lite 排行榜 [13]。该提交是首个仅使用开源模型就解决了 23.7%

⁴https://github.com/swe-bench/experiments/tree/main/evaluation/lite/20241016_IBM-SWE-1.0

```

1  defs:
2    demos:
3      data:
4        text:
5          |
6          ...
7
8      Question: Roger has 5 tennis balls.
9      He buys 2 more cans of tennis balls.
10     Each can has 3 tennis balls.
11     How many tennis balls does he have now?
12
13     Answer:
14     ...
15
16     text:
17     - "Roger started with\n"
18     - def: tennis_balls
19       data: 5
20     - "\ntennis ball.\n"
21     - "2 cans of 3 tennis balls each is\n"
22     - lang: python
23       def: bought_balls
24       code: result = 2 * 3
25     - "\ntennis balls.\n"
26     - "The result is: \n"
27     - lang: python
28       def: RESULT
29       code: result = ${ tennis_balls } + ${ bought_balls }
30
31   raw: true
32 text:
33 - model: watsonx/meta-llama/llama-3-70b-instruct
34   def: PDL
35   input:
36     text:
37       - ${ demos.text }
38       - "Question: ${ question }"
39   - lang: python
40     code: |
41       from pdl.pdl import exec_str
42       s = ""${ PDL }""
43       pdl = s.split("```")[1]
44       result = exec_str(pdl)
45   def: RESULT

```

James decides to run 1793815 sprints
1793815 times a week.
He runs 60 meters each sprint.
How many total meters does he run a week?

```

def solution():
    sprints_per_day = 1793815
    days_per_week = 3
    meters_per_sprint = 60
    total_sprints = sprints_per_day * days_per_week
    total_meters = total_sprints * meters_per_sprint
    result = total_meters
    return result

```

Fig. 6. GSMHard sample problematic data point

Fig. 5. PDL meta generation

Perhaps the most popular prompting framework today is LangChain [7], whose extensive features make it both powerful and complex. Avoiding this complexity is the main motivation for MiniChain [26], which offers fewer, simpler features that can be composed for advanced use-cases. However, both LangChain and MiniChain are Python frameworks, making them less declarative, since developers must write imperative code. PDL has a similar motivation to MiniChain, but goes a step further, by adopting YAML instead of Python as its foundation.

```

1  text:
2  - read: react_few_shot_samples.txt
3  - |
4
5  When was the discoverer of the Hudson River born?
6  - repeat:
7    text:
8    - def: thought
9      model: watsonx/ibm/granite-34b-code-instruct
10     parameters:
11       stop: ["Act:"]
12       include_stop_sequence: true
13    - def: action
14      model: watsonx/ibm/granite-34b-code-instruct
15      parameters:
16        stop: ["\n"]
17      parser: json
18      spec: {name: str, arguments: {topic: str}}
19    - def: observation
20      if: ${ action.name == "Search" }
21      then:
22        text:
23          - "Obs: "
24          - lang: python
25            code: |
26              import wikipedia
27              query = "${ action.arguments.topic }"
28              result = wikipedia.summary(query)
29      until: ${ action.name != "Search" }

```

科罗拉多造山带东段延伸区域的标高范围是多少？
思考：我需要搜索科罗拉多造山带，找到科罗拉多 [...] 东段延伸的区域
行动：{"name": "搜索", "arguments": {"topic": "科罗拉多造山带"}}
观察：科罗拉多造山带是一次 [...] 事件
[...]
Tho: I need to search the discoverer of the Hudson River, find when he was born.
Act: {"name": "Search", "arguments": {"topic": "discoverer of the Hudson River"}}
注：The Hudson River is a 315-mile [...]
Tho: The discoverer of the Hudson River is Henry Hudson. I need to search Henry Hudson, find when he was born.
Act: {"name": "Search", "arguments": {"topic": "Henry Hudson"}}
注：Henry Hudson (c. 1565 -disappeared [...]
Tho: Henry Hudson was born in 1565. Act: {"name": "Finish", "arguments": {"topic": "1565"}}

(a) 代码

(b) 解释器追踪

Fig. 4. PDL 中的 ReAct 智能体

个实例的方案，这一比例高于以往任何使用开源模型的结果，并且与前沿模型处于同一水平。

5.3 Generating PDL from PDL with LLMs

前几节展示了人类开发者如何使用 PDL 来编码不同的提示模式。本节转向大语言模型（LLMs），并展示它们如何也能用于生成 PDL。这种 PDL 元生成在 LLMs 需要为解决创建计划时很有帮助，例如作为智能体工作流的一部分。传统上，此类计划只是

Like other prompting frameworks, PDL aims to make LLMs less brittle. Guidance [22] is a Python-based framework that provides more structure but is more low-level than LangChain. Similarly, LMQL [5] is a domain-specific language embedded in Python that leverages types and constrained decoding. PDL takes some inspiration from the latter in how it intersperses prompts and programs, but unlike LMQL, relies less on imperative Python code. Crouse et al. use finite state machines to formally specify the internals of various agentic loops [9]; while fascinating, this work does not introduce a full-fledged prompting language.

One advantage of domain-specific languages is that they enable program transformation, e.g., for optimization [21]. The DSPy prompting framework [14] has the tag-line “programming, not prompting”: it finds prompts automatically so developers need not write them by hand. Similarly, Vieira [18] extends Prolog with LLMs as probabilistic relations, for which it automatically derives prompts. Both DSPy and Vieira are very high-level, but unlike PDL, both take away control from the developer over exact prompts. One language that lets users gradually adjust this tradeoff between automation and control for AI pipelines is Lale [3]; however, Lale does not focus on LLM prompting in particular. Whereas DSPy, Vieira, and Lale optimize for predictive performance, another objective to optimize for is computational performance. SGLang [34] does that by scheduling to better leverage prefix caching (to get more cache hits in a KV cache [16]). Future work will explore whether PDL’s declarative nature can enable similar computational performance optimizations.

Recently, a new crop of prompting framework has emerged that focuses on LLM-based agents. AutoGen [31] is a multi-agent framework where everything consists of agents and conversations. Other multi-agent frameworks include CrewAI [23] and GPSTSwarm [35]. These frameworks prioritize support for agents over support for other LLM-based use-cases. While PDL supports agents as well, it aims for a more balanced stance, where agents are just one of many prompting techniques.

7 Conclusion

PDL is a declarative data-oriented language: a program consists of YAML blocks, where each block either is a literal piece of data or produces data. The mental model is that executing a block appends its data to the background context, and subsequent LLM calls use that context as their prompt. This paper introduces the language via example programs and a tour of the grammar and tooling. The declarative nature of the language also makes it amenable to automatic optimizations for speed, accuracy, and security, which will be forthcoming in future work. PDL is ready to use and open-source at <https://github.com/IBM/prompt-declaration-language>.

References

- [1] Ibrahim Abdelaziz, Kinjal Basu, Mayank Agarwal, Sadhana Kumaravel, Matthew Stallone, Rameswar Panda, Yara Rizk, GP Bhargav, Maxwell Crouse, Chulaka Gunasekara, Shajith Ikbal, Sachin Joshi, Hima Karanam, Vineet Kumar, Asim Munawar, Sumit Neelam, Dinesh Raghu, Udit Sharma, Adriana Meza Soria, Dheeraj Sreedhar, Praveen Venkateswaran, Merve Unuvar, David Cox, Salim Roukos, Luis Lastras, and Pavan Kapanipathi. 2024. Granite-Function Calling Model: Introducing Function Calling Abilities via Multi-task Learning of Granular Tasks. <https://arxiv.org/abs/2407.00121>
- [2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. <https://arxiv.org/abs/2108.07732>
- [3] Guillaume Baudart, Martin Hirzel, Kiran Kate, Parikshit Ram, Avraham Shinnar, and Jason Tsay. 2021. Pipeline Combinators for Gradual AutoML. In *Advances in Neural Information Processing Systems (NeurIPS)*. 19705–19718. <https://proceedings.neurips.cc/paper/2021/file/a3b36cb25e2e0b93b5f334ffb4e4064e-Paper.pdf>
- [4] Oren Ben-Kiki, Clark Evans, and Brian Ingerson. 2004. YAML Ain’t Markup Language. <http://yaml.org/spec/history/2004-12-28/2004-12-28.pdf>
- [5] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2023. Prompting Is Programming: A Query Language for Large Language Models. In *Conference on Programming Language Design and Implementation (PLDI)*. 1946–1969. <https://doi.org/10.1145/3591300>

```

1  defs:
2    demos:
3      data:
4        text:
5          |
6          ...
7
8      Question: Roger has 5 tennis balls.
9      He buys 2 more cans of tennis balls.
10     Each can has 3 tennis balls.
11     How many tennis balls does he have now?
12
13     Answer:
14     ```
15
16     text:
17     - "Roger started with\n"
18     - def: tennis_balls
19       data: 5
20     - "\ntennis ball.\n"
21     - "2 cans of 3 tennis balls each is\n"
22     - lang: python
23       def: bought_balls
24       code: result = 2 * 3
25     - "\ntennis balls.\n"
26     - "The result is: \n"
27     - lang: python
28       def: RESULT
29       code: result = ${ tennis_balls } + ${ bought_balls }
30     ```
31
32     raw: true
33
34     text:
35     - model: watsonx/meta-llama/llama-3-70b-instruct
36
37     def: PDL
38
39     input:
40       text:
41         - ${ demos.text }
42         - "Question: ${ question }"
43
44     - lang: python
45       code: |
46         from pdl.pdl import exec_str
47
48         s = ""${ PDL }""
49         pdl = s.split("```")[1]
50         result = exec_str(pdl)
51
52     def: RESULT

```

Fig. 5. PDL 元生成

詹姆斯决定每周进行 1793815 次
1793815 趟短跑。
他每次短跑跑 60 米。
他每周总共跑多少米？

```

def solution():
    sprints_per_day = 1793815
    days_per_week = 3
    meters_per_sprint = 60
    total_sprints = sprints_per_day * days_per_week
    total_meters = total_sprints * meters_per_sprint
    result = total_meters
    return result

```

Fig. 6. GSMHard 样本问题数据点

- [6] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. <https://arxiv.org/abs/2005.14165>
- [7] Harrison Chase et al. 2022. LangChain. <https://github.com/langchain-ai/langchain>
- [8] Ezra Cooper, Sam Lindley, Philip Wadler, and Jeremy Yallop. 2006. Links: Web Programming Without Tiers. In *Symposium on Formal Methods for Components and Objects (FMCO)*. 266–296. https://doi.org/10.1007/978-3-540-74792-5_12
- [9] Maxwell Crouse, Ibrahim Abdelaziz, Ramon Astudillo, Kinjal Basu, Soham Dan, Sadhana Kumaravel, Achille Fokoue, Pavan Kapanipathi, Salim Roukos, and Luis Lastras. 2024. Formally Specifying the High-Level Behavior of LLM-Based Agents. <https://arxiv.org/abs/2310.08535>
- [10] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: Program-aided Language Models. In *International Conference on Machine Learning (ICML)*. 10764–10799. <https://proceedings.mlr.press/v202/gao23f.html>
- [11] Granite Team, IBM. 2024. Granite 3.0 Language Models. <https://github.com/ibm-granite/granite-3.0-language-models/blob/main/paper.pdf>
- [12] Hugging Face. 2023. Chat Templates. https://huggingface.co/docs/transformers/en/chat_templating
- [13] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=VTF8yNQm66>
- [14] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2023. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. <https://arxiv.org/abs/2310.03714>
- [15] Rahul Krishna, Rangeet Pan, Raju Pavuluri, Srikanth Tamilselvam, Maja Vukovic, and Saurabh Sinha. 2024. Codellm-Devkit: A Framework for Contextualizing Code LLMs with Program Analysis Insights. <https://arxiv.org/abs/2410.13007>
- [16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Symposium on Operating Systems Principles (SOSP)*. 611–626. <https://doi.org/10.1145/3600006.3613165>
- [17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Conference on Neural Information Processing Systems (NeurIPS)*. 9459–9474. <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>
- [18] Ziyang Li, Jiani Huang, Jason Liu, Felix Zhu, Eric Zhao, William Dodds, Neelay Velingker, Rajeev Alur, and Mayur Naik. 2024. Relational Programming with Foundation Models. In *Conference on Artificial Intelligence (AAAI)*. 10635–10644. <https://doi.org/10.1609/aaai.v38i9.28934>
- [19] Xiaoxia Liu, Jingyi Wang, Jun Sun, Xiaohan Yuan, Guoliang Dong, Peng Di, Wenhai Wang, and Dongxia Wang. 2023. Prompting Frameworks for Large Language Models: A Survey. <https://arxiv.org/abs/2311.12785>
- [20] John McCarthy. 1960. Recursive functions of symbolic expressions and their computation by machine, Part I. *Communications of the ACM (CACM)* 3, 4 (April 1960), 184–195. <https://doi.org/10.1145/367177.367199>
- [21] Marjan Mernik, Jan Heering, and Anthony M. Sloane. 2005. When and how to develop domain-specific languages. *ACM Computing Surveys (CSUR)* 37, 4 (2005), 316–344. <https://doi.org/10.1145/1118890.1118892>
- [22] Microsoft. 2023. [guidance]: A guidance language for controlling large language models. <https://github.com/langchain-ai/langchain>
- [23] João Moura. 2023. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. <https://github.com/crewAIInc/crewAI>
- [24] Felipe Pezoa, Juan L. Reutter, Fernando Suarez, Martín Ugarte, and Domagoj Vrgoč. 2016. Foundations of JSON Schema. In *International Conference on World Wide Web (WWW)*. 263–273. <https://doi.org/10.1145/2872427.2883029>
- [25] Armin Ronacher. 2008. Jinja2 Documentation Release 2.0. <http://mitsuhiko.pocoo.org/jinja2docs/Jinja2.pdf>
- [26] Alexander Rush. 2023. MiniChain: A Small Library for Coding with Large Language Models. In *Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP-Demo)*. 311–317. <https://aclanthology.org/2023.emnlp-demo.27/>
- [27] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems (NeurIPS)*. https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html

文本、JSON 或 Python 代码。使用 PDL，这些计划可以是完全可执行的模型调用和代码调用的组合。本节探讨将 PDL 元生成用于 GSMHard 数据集⁵。

GSMHard 是 GSM8k 的一个更难版本，后者包含需要简单算术或符号推理的小学数学问题。GSMHard 包含一个作为数学问题陈述的输入，以及一个作为解决问题 Python 代码的输出。我们实现了 PAL [10] 方法，但不是生成 Python 代码，而是要求 LLM 生成 PDL。文本形式的思维链被表示为 PDL 文本块，而算术运算则使用 PDL 代码块完成。

图 5 展示了一个生成 PDL 代码并在同一程序中执行它的 PDL 程序。变量 `demos` 保存着旨在教导模型如何生成 PDL 代码的少量样本。在第 32 行，一个模型调用块使用这些样本以及一个作为自由变量的 `question` 作为输入。结果是用于解决该问题的 PDL 程序。第 38 行提取该 PDL 程序并在 Python 中执行它。此程序应用于 GSMHard 数据集，其中 `question` 被填入输入问题。

该实验发现，GSMHard 数据集中有 10% 的数据实际上是不正确的，即其实况真值与所提问题不一致。图 6 展示了这种不一致的一个例子。使用 PDL 有助于这一发现，因为生成的 PDL 代码是人类可读的，因此我们能够轻松检查与实况真值不匹配的数据点，并发现实况真值在某些情况下是错误的。我们使用 LLM 覆盖了整个数据集，并系统地挑选出看似不一致的示例。然后我们手动过滤结果以去除误报，并识别出 10% 存在此问题的数据点。

6 Related Work

一项近期调查将 *prompting framework* 定义为一个管理层，它管理、简化并促进大型语言模型与用户、工具或其他模型之间的交互 [19]。该调查强调，提示框架的一个主要缺陷是其学习曲线日益陡峭。

或许当前最流行的提示框架是 LangChain [7]，其丰富的功能使其既强大又复杂。避免这种复杂性是 MiniChain [26] 的主要动机，它提供了更少、更简单的功能，这些功能可以组合用于高级用例。然而，LangChain 和 MiniChain 都是 Python 框架，这使得它们不那么声明式，因为开发者必须编写命令式代码。PDL 与 MiniChain 有相似的动机，但更进一步，采用 YAML 而非 Python 作为其基础。

与其他提示框架一样，PDL 旨在使大型语言模型不那么脆弱。Guidance [22] 是一个基于 Python 的框架，它提供了更多结构，但比 LangChain 更底层。类似地，LMQL [5] 是一种嵌入在 Python 中的领域特定语言，它利用类型和约束解码。PDL 在如何交织提示和程序方面从后者汲取了一些灵感，但与 LMQL 不同，它较少依赖命令式的 Python 代码。Crouse 等人使用有限状态机来形式化指定各种智能体循环的内部机制 [9]；尽管这项工作引人入胜，但它并未引入一个成熟的提示语言。

⁵<https://huggingface.co/datasets/reasoning-machines/gsm-hard>

- [28] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 9895–9901. <https://doi.org/10.18653/v1/2021.emnlp-main.779>
- [29] A. van Wijngaarden, B.J. Mailloux, J.E.L. Peck, C.H.A. Koster, M. Sintzoff, C.H. Lindsey, L.G.T. Meertens, and R.G. Fisker. 1977. Revised Report on the Algorithmic Language ALGOL 68. *ACM SIGPLAN Notices* 12, 5 (May 1977), 1–70. <https://doi.org/10.1145/954652.1781176>
- [30] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C. Schmidt. 2023. A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. <https://arxiv.org/abs/2302.11382>
- [31] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryan W White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. <https://arxiv.org/abs/2308.08155>
- [32] Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, and Dongkuan Xu. 2023. Decoupling Reasoning from Observations for Efficient Augmented Language Models. <https://openreview.net/forum?id=CpgoO6j6W1>
- [33] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=WE_vluYUL-X
- [34] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2023. Efficiently Programming Large Language Models using SGLang. <https://arxiv.org/abs/2312.07104>
- [35] Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. 2024. GPTSwarm: Language Agents as Optimizable Graphs. In *International Conference on Machine Learning (ICML)*. <https://openreview.net/forum?id=uTC9AFXIhg>

领域特定语言的一个优势是它们支持程序转换，例如用于优化 [21]。DSPy 提示框架 [14] 的口号是“编程，而非提示”：它能自动找到提示，因此开发者无需手动编写。类似地，Vieira [18] 将 Prolog 扩展为包含作为概率关系的大型语言模型，并自动为其推导提示。DSPy 和 Vieira 都非常高层，但与 PDL 不同，两者都剥夺了开发者对精确提示的控制权。Lale [3] 是一种让用户逐步调整 AI 流程中自动化与控制之间权衡的语言；然而，Lale 并不特别专注于大型语言模型提示。尽管 DSPy、Vieira 和 Lale 针对预测性能进行优化，另一个可优化的目标是计算性能。SGLang [34] 通过调度来更好地利用前缀缓存（以在 KV 缓存中获得更多缓存命中 [16]）来实现这一点。未来的工作将探讨 PDL 的声明式特性是否能够实现类似的计算性能优化。

最近，一批新的提示框架已经出现，专注于基于大型语言模型的智能体。AutoGen [31] 是一个多智能体框架，其中所有内容都由智能体和对话构成。其他多智能体框架包括 CrewAI [23] 和 GPTSwarm [35]。这些框架优先支持智能体，而非其他基于大型语言模型的用例。虽然 PDL 也支持智能体，但它追求一种更平衡的立场，即智能体只是众多提示技术中的一种。

7 Conclusion

PDL 是一种声明式数据导向语言：程序由 YAML 块组成，每个块要么是字面数据片段，要么生成数据。其心智模型是：执行一个块会将其数据追加到后台上下文中，后续的 LLM 调用将该上下文用作其提示。本文通过示例程序和语法工具链导览介绍该语言。该语言的声明式特性也使其易于进行速度、准确性和安全性的自动优化，这些优化将在未来工作中实现。PDL 已可投入使用，其开源地址为 <https://github.com/IBM/prompt-declaration-language>。

References

- [1] Ibrahim Abdelaziz, Kinjal Basu, Mayank Agarwal, Sadhana Kumaravel, Matthew Stallone, Rameswar Panda, Yara Rizk, GP Bhargav, Maxwell Crouse, Chulaka Gunasekara, Shajith Ikbal, Sachin Joshi, Hima Karanam, Vineet Kumar, Asim Munawar, Sumit Neelam, Dinesh Raghu, Udit Sharma, Adriana Meza Soria, Dheeraj Sreedhar, Praveen Venkateswaran, Merve Unuvar, David Cox, Salim Roukos, Luis Lastras, and Pavan Kapanipathi. 2024. Granite-Function Calling Model: Introducing Function Calling Abilities via Multi-task Learning of Granular Tasks. <https://arxiv.org/abs/2407.00121>
- [2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. <https://arxiv.org/abs/2108.07732>
- [3] Guillaume Baudart, Martin Hirzel, Kiran Kate, Parikshit Ram, Avraham Shinnar, and Jason Tsay. 2021. Pipeline Combinators for Gradual AutoML. In *Advances in Neural Information Processing Systems (NeurIPS)*. 19705–19718. <https://proceedings.neurips.cc/paper/2021/file/a3b36cb25e2e0b93b5f334ffb4e4064e-Paper.pdf>
- [4] Oren Ben-Kiki, Clark Evans, and Brian Ingerson. 2004. YAML Ain't Markup Language. <http://yaml.org/spec/history/2004-12-28/2004-12-28.pdf>
- [5] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2023. Prompting Is Programming: A Query Language for Large Language Models. In *Conference on Programming Language Design and Implementation (PLDI)*. 1946–1969. <https://doi.org/>

- 10.1145/3591300
- [6] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Matusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. <https://arxiv.org/abs/2005.14165>
 - [7] Harrison Chase et al. 2022. LangChain. <https://github.com/langchain-ai/langchain>
 - [8] Ezra Cooper, Sam Lindley, Philip Wadler, and Jeremy Yallop. 2006. Links: Web Programming Without Tiers. In *Symposium on Formal Methods for Components and Objects (FMCO)*. 266–296. https://doi.org/10.1007/978-3-540-74792-5_12
 - [9] Maxwell Crouse, Ibrahim Abdelaziz, Ramon Astudillo, Kinjal Basu, Soham Dan, Sadhana Kumaravel, Achille Fokoue, Pavan Kapanipathi, Salim Roukos, and Luis Lastras. 2024. Formally Specifying the High-Level Behavior of LLM-Based Agents. <https://arxiv.org/abs/2310.08535>
 - [10] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: Program-aided Language Models. In *International Conference on Machine Learning (ICML)*. 10764–10799. <https://proceedings.mlr.press/v202/gao23f.html>
 - [11] Granite Team, IBM. 2024. Granite 3.0 Language Models. <https://github.com/ibm-granite/granite-3.0-language-models/blob/main/paper.pdf>
 - [12] Hugging Face. 2023. Chat Templates. https://huggingface.co/docs/transformers/en/chat_templating
 - [13] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=VTF8yNQm66>
 - [14] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2023. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. <https://arxiv.org/abs/2310.03714>
 - [15] Rahul Krishna, Rangeet Pan, Raju Pavuluri, Srikanth Tamilselvam, Maja Vukovic, and Saurabh Sinha. 2024. Codellm-Devkit: A Framework for Contextualizing Code LLMs with Program Analysis Insights. <https://arxiv.org/abs/2410.13007>
 - [16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Symposium on Operating Systems Principles (SOSP)*. 611–626. <https://doi.org/10.1145/3600006.3613165>
 - [17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Conference on Neural Information Processing Systems (NeurIPS)*. 9459–9474. <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>
 - [18] Ziyang Li, Jiani Huang, Jason Liu, Felix Zhu, Eric Zhao, William Dodds, Neelay Velingker, Rajeev Alur, and Mayur Naik. 2024. Relational Programming with Foundation Models. In *Conference on Artificial Intelligence (AAAI)*. 10635–10644. <https://doi.org/10.1609/aaai.v38i9.28934>
 - [19] Xiaoxia Liu, Jingyi Wang, Jun Sun, Xiaohan Yuan, Guoliang Dong, Peng Di, Wenhai Wang, and Dongxia Wang. 2023. Prompting Frameworks for Large Language Models: A Survey. <https://arxiv.org/abs/2311.12785>
 - [20] John McCarthy. 1960. Recursive functions of symbolic expressions and their computation by machine, Part I. *Communications of the ACM (CACM)* 3, 4 (April 1960), 184–195. <https://doi.org/10.1145/367177.367199>
 - [21] Marjan Mernik, Jan Heering, and Anthony M. Sloane. 2005. When and how to develop domain-specific languages. *ACM Computing Surveys (CSUR)* 37, 4 (2005), 316–344. <https://doi.org/10.1145/1118890.1118892>
 - [22] Microsoft. 2023. {guidance}: A guidance language for controlling large language models. <https://github.com/langchain-ai/langchain>
 - [23] João Moura. 2023. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. <https://github.com/crewAIInc/crewAI>
 - [24] Felipe Pezosa, Juan L. Reutter, Fernando Suarez, Martín Ugarte, and Domagoj Vrgoč. 2016. Foundations of JSON Schema. In *International Conference on World Wide Web (WWW)*. 263–273. <https://doi.org/10.1145/2872427.2883029>
 - [25] Armin Ronacher. 2008. Jinja2 Documentation Release 2.0. <http://mitsuhiko.pocoo.org/jinja2docs/Jinja2.pdf>

- [26] Alexander Rush. 2023. MiniChain: A Small Library for Coding with Large Language Models. In *Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP-Demo)*. 311–317. <https://aclanthology.org/2023.emnlp-demo.27/>
- [27] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems (NeurIPS)*. https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html
- [28] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 9895–9901. <https://doi.org/10.18653/v1/2021.emnlp-main.779>
- [29] A. van Wijngaarden, B.J. Mailloux, J.E.L. Peck, C.H.A. Koster, M. Sintzoff, C.H. Lindsey, L.G.T. Meertens, and R.G. Fisker. 1977. Revised Report on the Algorithmic Language ALGOL 68. *ACM SIGPLAN Notices* 12, 5 (May 1977), 1–70. <https://doi.org/10.1145/954652.1781176>
- [30] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C. Schmidt. 2023. A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. <https://arxiv.org/abs/2302.11382>
- [31] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. <https://arxiv.org/abs/2308.08155>
- [32] Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, and Dongkuan Xu. 2023. Decoupling Reasoning from Observations for Efficient Augmented Language Models. <https://openreview.net/forum?id=CpgO6j6W1>
- [33] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=WE_vluYUL-X
- [34] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2023. Efficiently Programming Large Language Models using SGLang. <https://arxiv.org/abs/2312.07104>
- [35] Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. 2024. GPTSwarm: Language Agents as Optimizable Graphs. In *International Conference on Machine Learning (ICML)*. <https://openreview.net/forum?id=uTC9AFXIhg>